

Project Files Structure - 001

Ordoñez Silva, Yonel Jr.

Indice

I. Introducción	4
II. Marco Teórico	4
2.1. Modelo C4 o Diagrama C4	4
III. Visión General	4
3.1. Diagrama C4 - General y Backend	5
3.2. Diagrama de Despliegue	7
IV. Domain Layer	7
Diagrama C4	10
Análisis de Cada Componente	11
Exceptions/	11
DomainException.cs	11
ValueObjects/	11
AnimalId.cs	11
AnimalName.cs	11
Enum/	12
AnimalStatus.cs	12
Rules	12
AnimalRules.cs	12
Entities/	13
Animal.cs	13
V. Application Layer	14
Diagrama C4	16
Análisis de Cada Componente	17
Common/	17
Gateway/	17
Context/	17

Caching/	17
Configuration/	18
Features/	18
Rules/	18
Identity/	18
Repositories/	19
Time/	19
Behaviors/	19
IPipelineBehavior.cs	20
ValidationBehavior.cs	20
LoggingBehavior.cs	20
Exceptions/	21
BusinessException.cs	21
Models/	21
PaginationModel.cs	21
Modules/	21
Animals/UseCases/CreateAnimal	21
CreateAnimalCommand.cs	22
CreateAnimalOutput.cs	22
ICreateAnimalInputPort.cs	22
ICreateAnimalOutputPort.cs	22
CreateAnimalInteractor.cs	23
CreateAnimalValidator.cs	25
VI. InterfaceAdapters Layer	26
Diagrama C4	27
Análisis de Cada Componente	27
DTOs/	27
Requests/	27
CreateAnimalRequest.cs	27
Responses/	27
AnimalResponse.cs	28
Mappers/	28
AnimalMapper.cs	28
Presenters/	28
CreateAnimalPresenter.cs	28
Controllers/	29
AnimalsController.cs	29
Middleware/	30
ExceptionHandlingMiddleware.cs	30
TenantResolutionMiddleware.cs	31
Filters/	32
ValidationFilter.cs	32

VII. Infrastructure Layer	33
Diagrama C4	36
Análisis de Cada Componente	37
Persistence/	37
GanaderiaDbContext.cs	37
TenantCatalogDb.cs	38
Entities/	38
CowEntity.cs	38
Mappers/	39
CowMapper.cs	39
Configurations/	39
CowConfiguration.cs	39
Repositories/	40
AnimalRepository.cs	40
Tenant/	41
ITenantDbContextFactory.cs	41
TenantDbContextFactory.cs	42
TenantContext.cs	43
ITenantStore.cs	43
TenantInfo.cs	43
TenantStore.cs	44
TenantProvisioningService.cs	45
Configuration/	47
ConfigurationService.cs	47
Features/	48
FeatureService.cs	48
Rules/	49
RuleEngineService.cs	49
Caching/	50
AppCacheService.cs	50
Context/	51
RequestContext.cs	51
Identity/	51
CurrentUserService.cs	51
Auditing/	52
AuditSaveChangesInterceptor.cs	52
Time/	53
DateTimeProvider.cs	53
External/	54
SenasaClient.cs	54
FLUJO COMPLETO	54

I. Introducción

Este documento describe en detalle **cada capa, componente, carpeta y archivo** de la arquitectura basada en Clean Architecture + enfoque modular + multi-tenant.

II. Marco Teórico

2.1. Modelo C4 o Diagrama C4

Es un conjunto de cuatro diagramas jerárquicos (Context, Contenedores, Componentes y Código) diseñado para representar la arquitectura de software, actuando como un mapa con diferentes niveles de detalle.

III. Visión General

```
InterfaceAdapters → Application → Domain
                    ↑
                Infrastructure
```

- **Domain:** reglas del negocio
- **Application:** orquestación (casos de uso)
- **InterfaceAdapters:** entrada/salida (HTTP, UI)
- **Infrastructure:** implementación técnica

3.1. Diagrama C4 - General y Backend

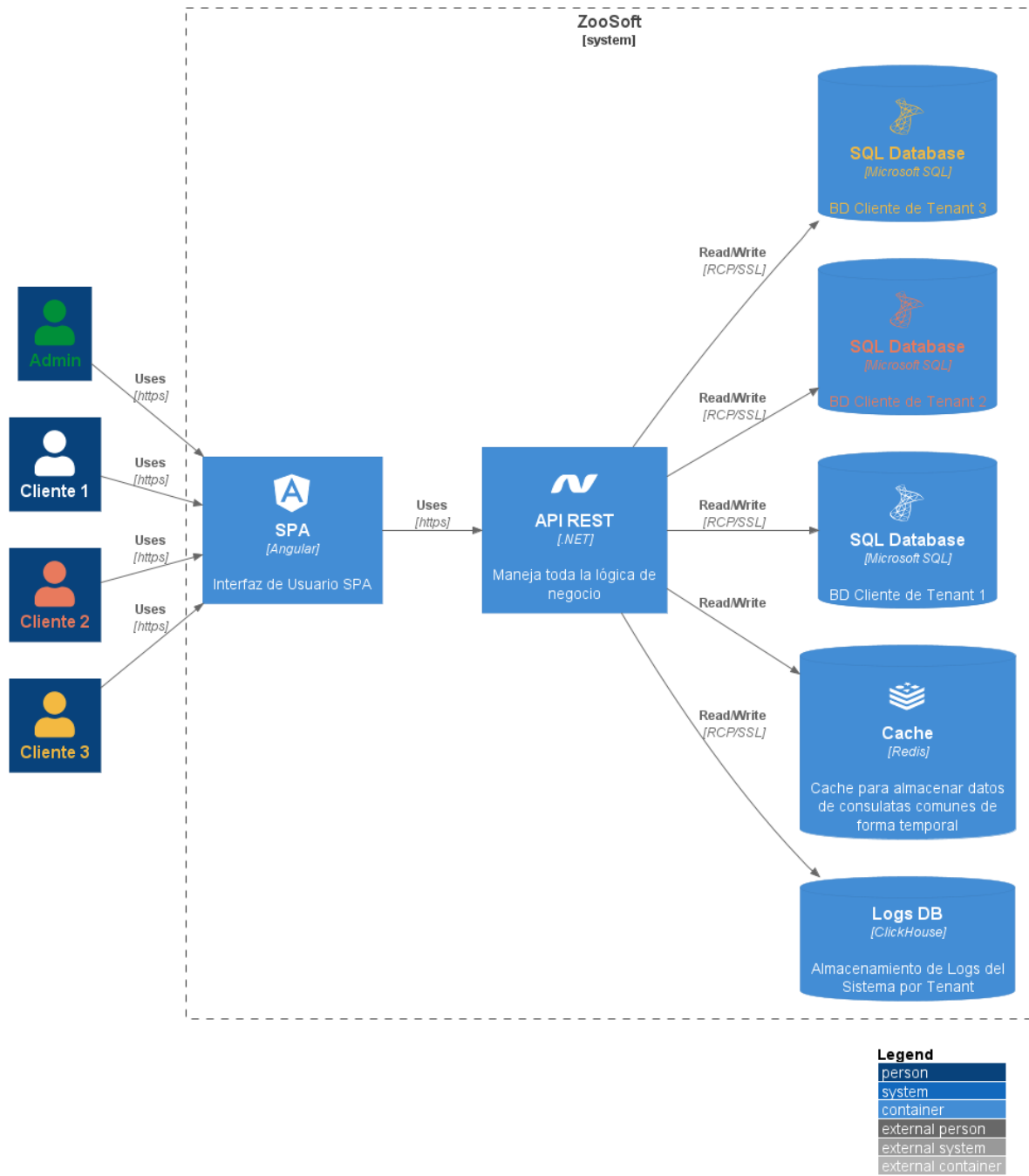


Figure 1: Diagrama C4 - General

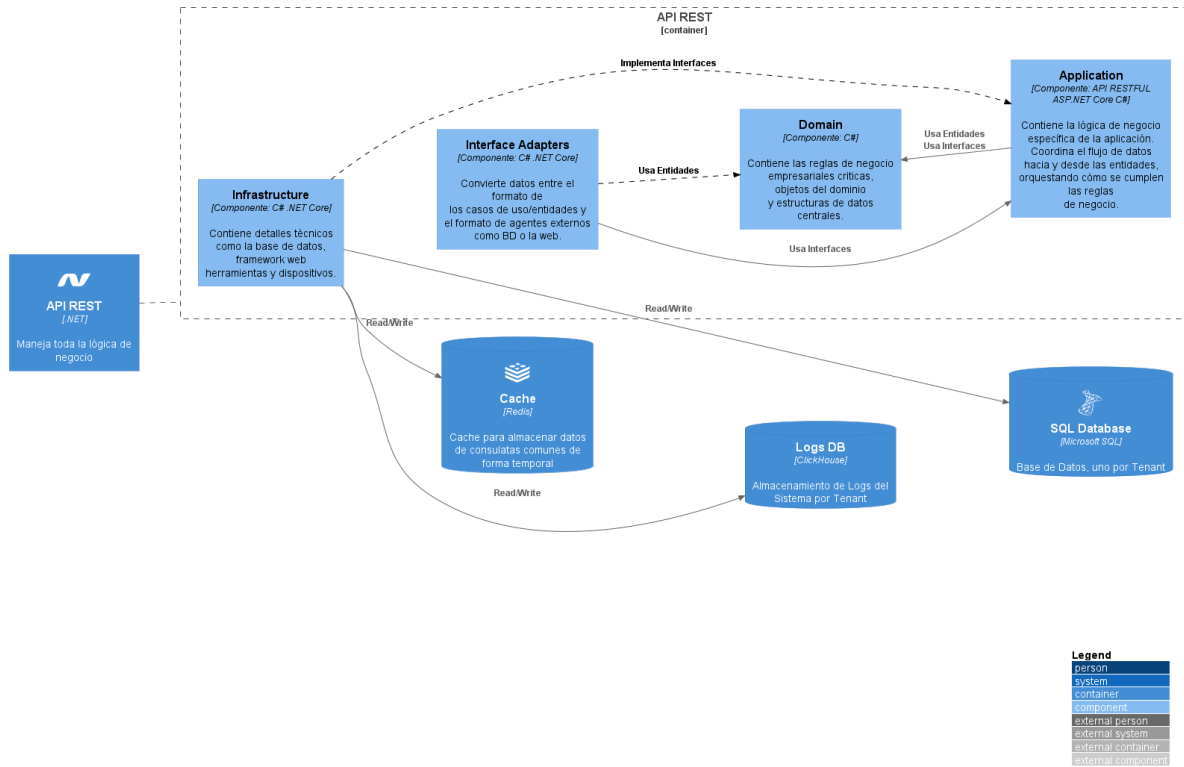


Figure 2: Diagrama C4 - Backend

3.2. Diagrama de Despliegue

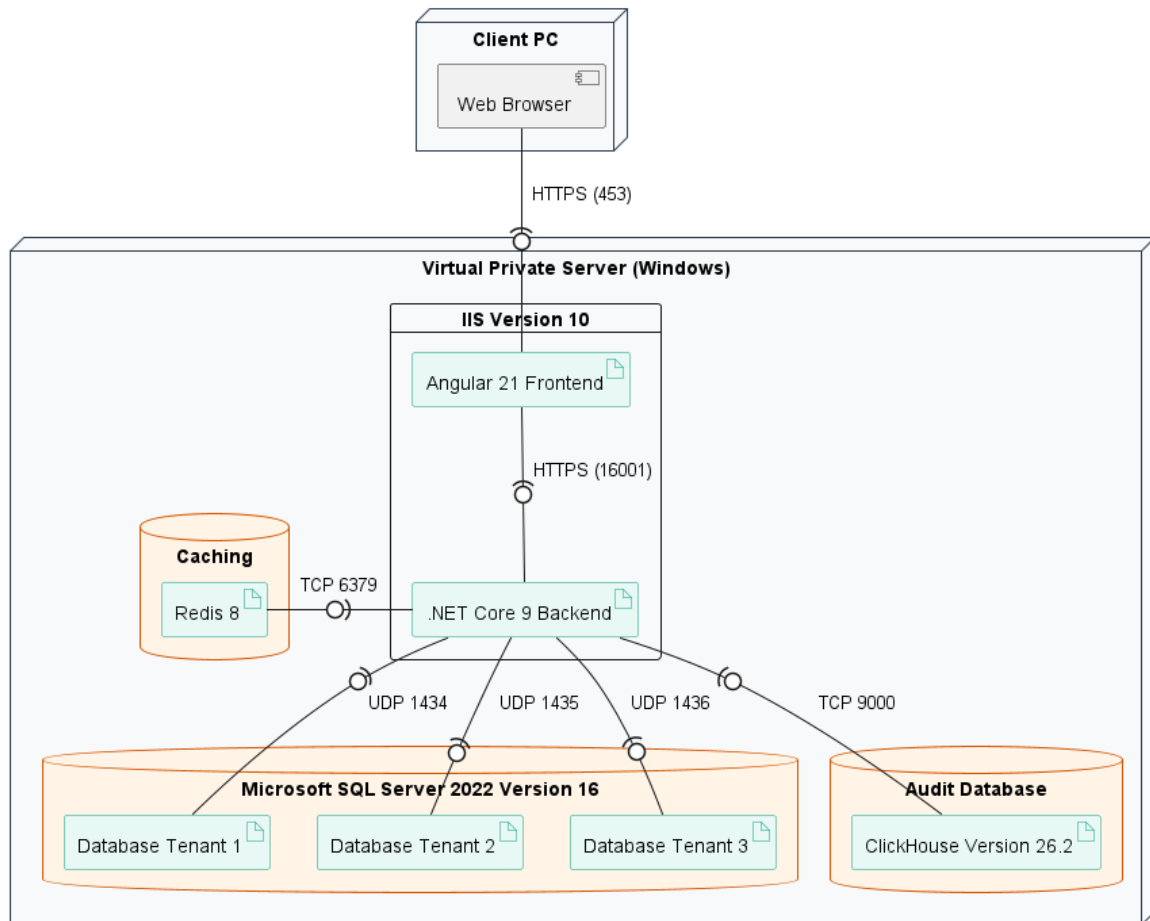


Figure 3: Diagrama de Despliegue

IV. Domain Layer

```
Domain/  
  Entities/  
    Product.cs  
    Animal.cs
```

```
MilkRecord.cs
ReproductionEvent.cs
AuditEntry.cs

ValueObjects/
  AnimalId.cs
  Weight.cs
  MilkQuantity.cs

Enums/
  AnimalStatus.cs
  ReproductionType.cs

Rules/
  BusinessRule.cs

Exceptions/
  DomainException.cs

Services/
  (opcional: lógica compleja de dominio)
```


Diagrama C4

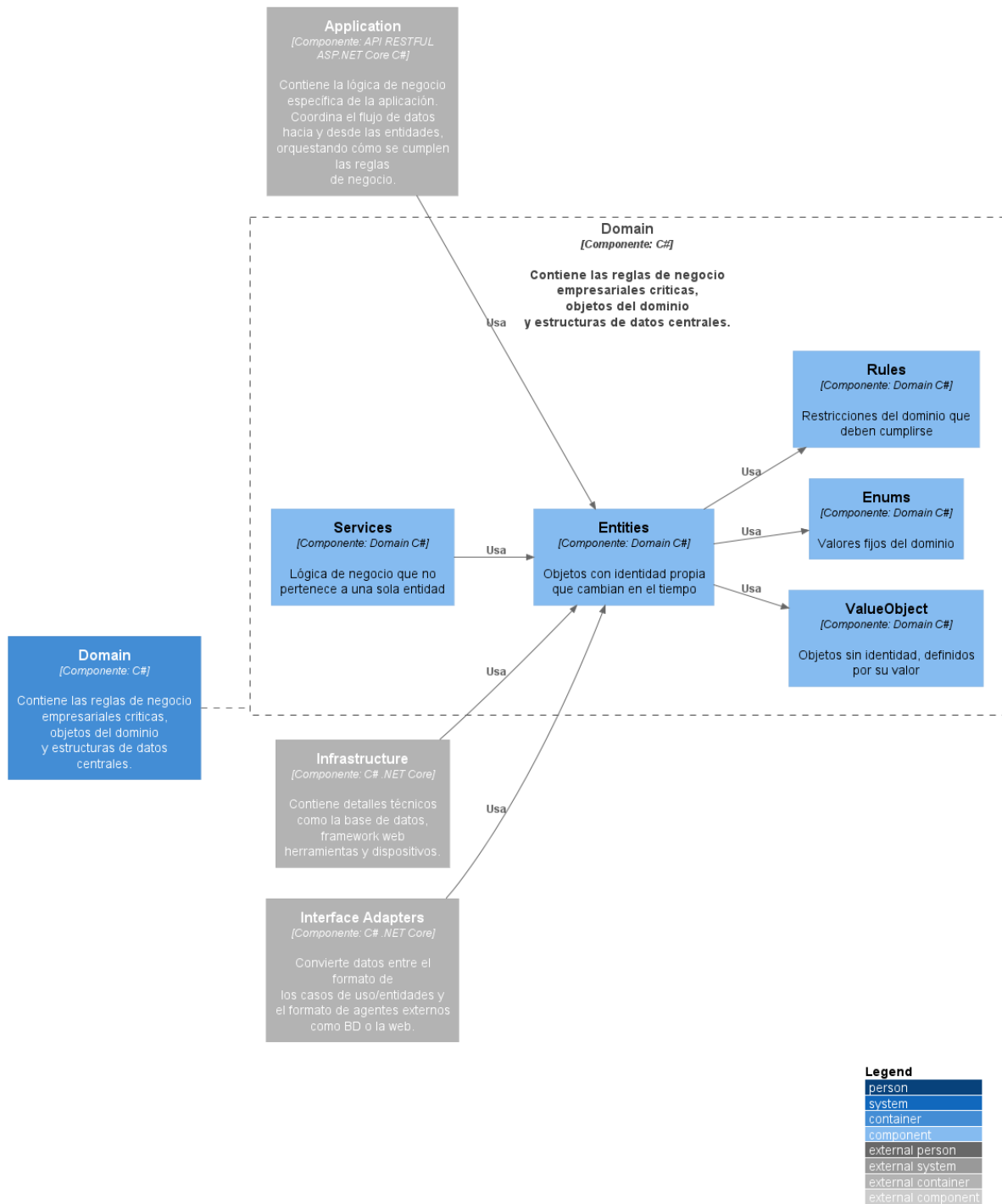


Figure 4: Diagrama C4 - Domain

Análisis de Cada Componente

Exceptions/

DomainException.cs

```
namespace Domain.Exceptions;

// Base para errores de negocio
public class DomainException : Exception
{
    public DomainException(string message) : base(message) { }
}
```

ValueObjects/

AnimalId.cs

```
namespace Domain.ValueObjects;

public class AnimalId
{
    public Guid Value { get; }

    public AnimalId(Guid value)
    {
        if (value == Guid.Empty)
            throw new ArgumentException("Id inválido");

        Value = value;
    }
}
```

AnimalName.cs

```
using Domain.Exceptions;

namespace Domain.ValueObjects;

public class AnimalName
{
}
```

```

public string Value { get; }

public AnimalName(string value)
{
    if (string.IsNullOrEmpty(value))
        throw new DomainException("El nombre es obligatorio");

    if (value.Length < 3)
        throw new DomainException("El nombre es demasiado corto");

    Value = value;
}
}

```

Enum/

AnimalStatus.cs

```

namespace Domain.Enums;

public enum AnimalStatus
{
    Active,
    Inactive,
    Sold
}

```

Rules

AnimalRules.cs

```

using Domain.Exceptions;

namespace Domain.Rules;

public static class AnimalRules
{
    public static void ValidateBirthDate(DateTime birthDate)
    {
        if (birthDate > DateTime.UtcNow)

```

```

        throw new DomainException("La fecha de nacimiento no puede ser futura");
    }
}

```

Entities/

Animal.cs

```

using Domain.ValueObjects;
using Domain.Enums;
using Domain.Rules;

namespace Domain.Entities;

public class Animal
{
    public Guid Id { get; private set; }
    public string Name { get; private set; }
    public DateTime BirthDate { get; private set; }
    public DateTime CreatedAt { get; private set; }
    public string? CreatedBy { get; private set; }
    public AnimalStatus Status { get; private set; }

    // Constructor privado → obliga a usar métodos controlados
    private Animal() { }

    // Factory method (RECOMENDADO)
    public static Animal Create(
        AnimalId id,
        AnimalName name,
        DateTime birthDate,
        DateTime createdAt,
        string? createdBy)
    {
        // Reglas de dominio
        AnimalRules.ValidateBirthDate(birthDate);

        return new Animal
        {
            Id = id.Value,
            Name = name.Value,

```

```

        BirthDate = birthDate,
        CreatedAt = createdAt,
        CreatedBy = createdBy,
        Status = AnimalStatus.Active
    };
}

// Comportamiento (NO setters públicos)
public void Deactivate()
{
    if (Status == AnimalStatus.Inactive)
        throw new Exception("El animal ya está inactivo");

    Status = AnimalStatus.Inactive;
}
}

```

V. Application Layer

```

Application/
    Common/
        Gateway/
            Context/
                IRequestContext.cs
                ITenantContext.cs

            Caching/
                IAppCacheService.cs

            Configuration/
                IConfigurationService.cs

            Features/
                IFeatureService.cs

            Rules/

```

```
        IRuleEngineService.cs

    Identity/
        ICurrentUserService.cs

    Repositories/
        IAnimalRepository.cs

    Time/
        IDateTimeProvider.cs

    Behaviors/
    Exceptions/
    Models/

Modules/

    Animals/
        UseCases/
            CreateAnimal/
                Ports/
                    ICreateAnimalInputPort.cs
                    ICreateAnimalOutputPort.cs

                    CreateAnimalInteractor.cs
                    CreateAnimalCommand.cs
                    CreateAnimalOutput.cs
                    CreateAnimalValidator.cs

        DTOs/

    Reproduction/
        UseCases/
        Interfaces/
            IReproductionRepository.cs

    Production/
        UseCases/
        Interfaces/
            IProductionRepository.cs

    Health/
```

UseCases/
Interfaces/
IHealthRepository.cs

DependencyInjection.cs

Diagrama C4

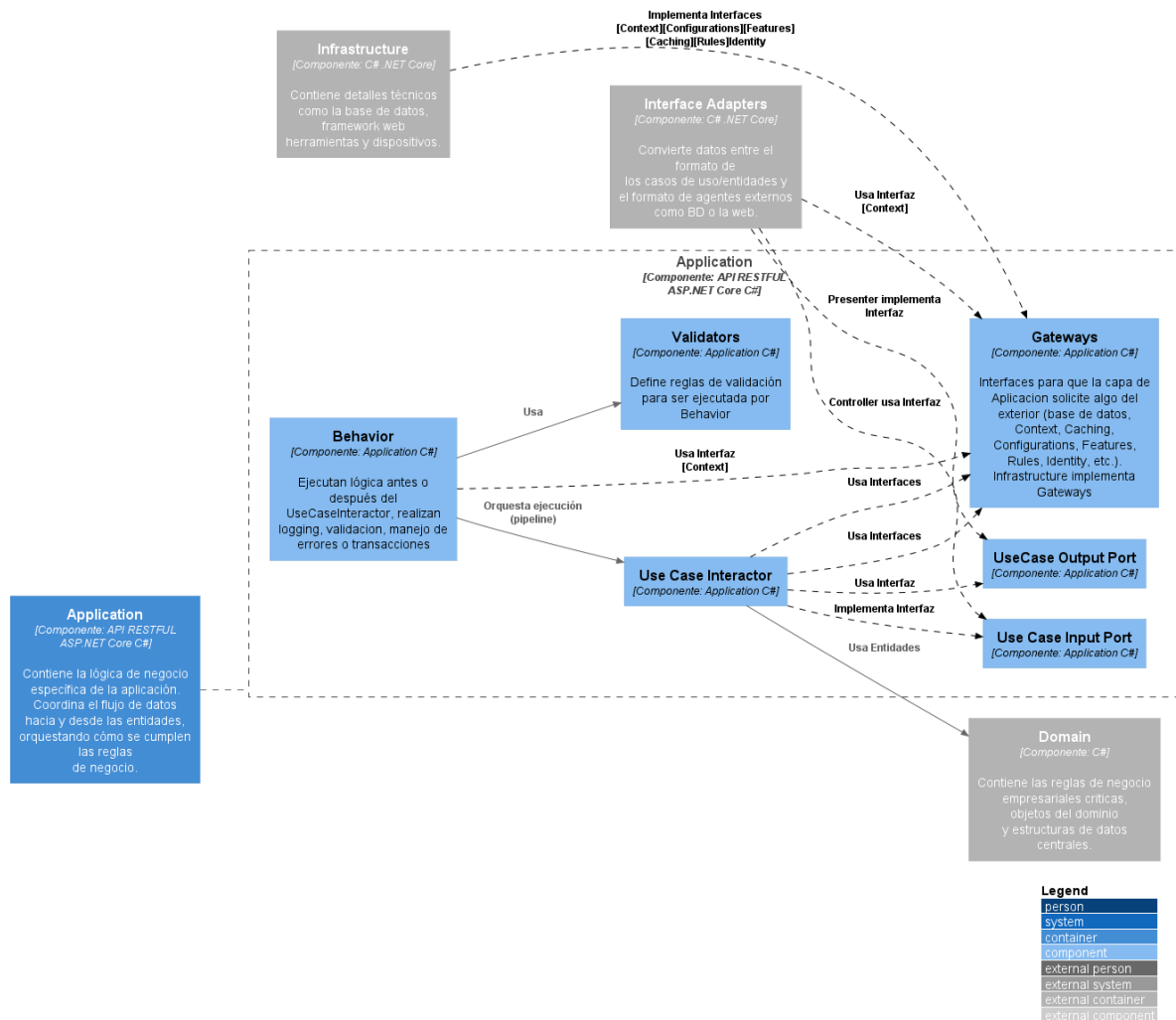


Figure 5: Diagrama C4 - Application

Análisis de Cada Componente

Common/

Gateway/

Context/

ITenantContext.cs

```
namespace Application.Common.Gateway.Context;

public interface ITenantContext
{
    Guid TenantId { get; }
    string DatabaseName { get; }
}
```

IRequestContext.cs

```
namespace Application.Common.Gateway.Context;

public interface IRequestContext
{
    string? UserId { get; }
    string? UserEmail { get; }
    string? CorrelationId { get; }
    string? Path { get; }
}
```

Caching/

IAppCacheService.cs

```
namespace Application.Common.Gateway.Caching;

// Abstracción de cache (NO depende de IMemoryCache)
public interface IAppCacheService
{
    Task<T> GetOrCreateAsync<T>(
        string key,
        Func<Task<T>> factory,
    )
```

```
        TimeSpan expiration);  
    }
```

Configuration/

IConfigurationService.cs

```
namespace Application.Common.Gateway.Configuration;  
  
public interface IConfigurationService  
{  
    Task<T?> GetValueAsync<T>(string key);  
}
```

Features/

IFeatureService.cs

```
namespace Application.Common.Gateway.Features;  
  
public interface IFeatureService  
{  
    Task<bool> IsEnabledAsync(string feature);  
}
```

Rules/

IRuleEngineService.cs

```
namespace Application.Common.Gateway.Rules;  
  
public interface IRuleEngineService  
{  
    Task<bool> EvaluateAsync(string rule, object input);  
}
```

Identity/

ICurrentUserService.cs

```
namespace Application.Common.Gateway.Identity;

public interface ICurrentUserService
{
    string? UserId { get; }
}
```

Repositories/

IAnimalRepository.cs

```
using Domain.Entities;

namespace Application.Common.Gateway.Repositories;

public interface IAnimalRepository
{
    Task AddAsync(Animal animal);
}
```

Time/

Permite abstraer el tiempo.

IDateTimeProvider.cs

```
namespace Application.Common.Gateway.Time;

public interface IDateTimeProvider
{
    DateTime.UtcNow { get; }
}
```

Behaviors/

Los Behaviors son un pipeline alrededor del UseCase, parecidos a los Middlewares en la capa de InterfaceAdapters. Se pueden llamar directamente antes de una request o colocar un pipeline para llamar al Behavior y justo después de su lógica llamar al UseCaseInteractor.

IPipelineBehavior.cs

```
namespace Application.Common.Behaviors;

public interface IPipelineBehavior<TRequest>
{
    Task Handle(TRequest request, Func<Task> next);
}
```

ValidationBehavior.cs

```
namespace Application.Common.Behaviors;

public class ValidationBehavior<TRequest> : IPipelineBehavior<TRequest>
{
    private readonly IEnumerable<IValidator<TRequest>> _validators;

    public ValidationBehavior(IEnumerable<IValidator<TRequest>> validators)
    {
        _validators = validators;
    }

    public async Task Handle(TRequest request, Func<Task> next)
    {
        foreach (var validator in _validators)
        {
            validator.Validate(request);
        }

        await next();
    }
}
```

LoggingBehavior.cs

```
namespace Application.Common.Behaviors;

public class LoggingBehavior<TRequest> : IPipelineBehavior<TRequest>
{
    public async Task Handle(TRequest request, Func<Task> next)
    {

```

```

        Console.WriteLine($"[LOG] Handling {typeof(TRequest).Name}");

        await next();

        Console.WriteLine($"[LOG] Completed {typeof(TRequest).Name}");
    }
}

```

Exceptions/

BusinessException.cs

```

namespace Application.Common.Exceptions;

public class BusinessException : Exception
{
    public BusinessException(string message) : base(message) { }
}

// Uso en UseCaseInteractor
if (!await _features.IsEnabledAsync("animals.create"))
    throw new BusinessException("Feature deshabilitada");

```

Models/

Aquí van modelos compartidos entre `UseCases` y estructuras complejas (filtros, paginación, etc.).

PaginationModel.cs

```

public class PaginationModel
{
    public int Page { get; set; }
    public int PageSize { get; set; }
}

```

Modules/

Animals/UseCases/CreateAnimal

CreateAnimalCommand.cs

Este es un DTO para mapear la solicitud del usuario.

```
namespace Application.Modules.Animals.UseCases.CreateAnimal;

public class CreateAnimalCommand
{
    public string Name { get; set; } = default!;
    public DateTime BirthDate { get; set; }
}
```

CreateAnimalOutput.cs

Este es otro DTO pero para mapear el response hacia el usuario.

```
namespace Application.Modules.Animals.UseCases.CreateAnimal;

public class CreateAnimalOutput
{
    public Guid Id { get; set; }
    public string Name { get; set; } = default!;
}
```

ICreateAnimalInputPort.cs

```
namespace Application.Modules.Animals.UseCases.CreateAnimal;

public interface ICreateAnimalInputPort
{
    Task Handle(CreateAnimalCommand command);
}
```

ICreateAnimalOutputPort.cs

```
public interface ICreateAnimalOutputPort
{
    Task Ok(CreateAnimalOutput output);
}
```

CreateAnimalInteractor.cs

```
using Domain.Entities;
using Application.Common.Exceptions;
using Application.Modules.Animals.Interfaces;
using Application.Common.Gateway.Features;
using Application.Common.Gateway.Configuration;
using Application.Common.Gateway.Rules;
using Application.Common.Gateway.Identity;
using Application.Common.Gateway.Time;
using Application.Common.Gateway.Caching;
using Application.Common.Gateway.Context;

namespace Application.Modules.Animals.UseCases.CreateAnimal;

public class CreateAnimalInteractor : ICreateAnimalInputPort
{
    private readonly IAnimalRepository _repo;
    private readonly ICreateAnimalOutputPort _output;
    private readonly IFeatureService _features;
    private readonly IConfigurationService _config;
    private readonly IRuleEngineService _rules;
    private readonly ICurrentUserService _user;
    private readonly IDateTimeProvider _time;
    private readonly IAppCacheService _cache;
    private readonly ITenantContext _tenant;

    public CreateAnimalInteractor(
        IAnimalRepository repo,
        ICreateAnimalOutputPort output,
        IFeatureService features,
        IConfigurationService config,
        IRuleEngineService rules,
        ICurrentUserService user,
        IDateTimeProvider time,
        IAppCacheService cache,
        ITenantContext tenant)
    {
        _repo = repo;
        _output = output;
        _features = features;
        _config = config;
        _rules = rules;
    }
}
```

```

        _user = user;
        _time = time;
        _cache = cache;
        _tenant = tenant;
    }

    public async Task Handle(CreateAnimalCommand cmd)
    {
        // Logging contextual
        Console.WriteLine($"User: {_request.UserId}");
        Console.WriteLine($"Path: {_request.Path}");
        Console.WriteLine($"CorrelationId: {_request.CorrelationId}");

        // 1. Feature
        if (!await _features.IsEnabledAsync("animals.create"))
            throw new BusinessException("Feature deshabilitada");

        // 2. Obtener tenant actual
        var tenantId = _tenant.TenantId;

        // 3. Usar cache POR TENANT
        var minNameLength = await _cache.GetOrCreateAsync(
            $"tenant:{tenantId}:config:minName",
            async () => await _config.GetValueAsync<int>("animal.name.min"),
            TimeSpan.FromMinutes(5)
        );

        // 4. Config (con cache)
        var minNameLength = await _cache.GetOrCreateAsync(
            $"config:minName:{_tenant.TenantId}",
            async () => await _config.GetValueAsync<int>("animal.name.min"),
            TimeSpan.FromMinutes(5)
        );

        if (cmd.Name.Length < minNameLength)
            throw new BusinessException("Nombre inválido");

        // 5. Rules
        var isValid = await _rules.EvaluateAsync(
            "animal.creation.allowed",
            new { cmd.Name });
    }
}

```

```

        if (!isValid)
            throw new BusinessException("Regla no cumplida");

        // 6. Crear entidad
        var animal = new Animal(
            Guid.NewGuid(),
            cmd.Name,
            cmd.BirthDate,
            _time.UtcNow,           // usando TimeProvider
            _user.UserId           // usando identidad
        );

        // 7. Persistencia
        await _repo.AddAsync(animal);

        // 8. Output
        await _output.Ok(new CreateAnimalOutput
        {
            Id = animal.Id,
            Name = animal.Name
        });
    }
}

```

CreateAnimalValidator.cs

El Validator contiene lógica que será validada por Behavior.

```

namespace Application.Modules.Animals.UseCases.CreateAnimal;

public class CreateAnimalValidator
{
    public void Validate(CreateAnimalCommand cmd)
    {
        if (string.IsNullOrEmpty(cmd.Name))
            throw new Exception("Nombre requerido");
    }
}

```

VI. InterfaceAdapters Layer

```
InterfaceAdapters/  
  Controllers/  
    ProductsController.cs  
    AnimalsController.cs  
    ReportsController.cs  
  
  DTOs/  
    Requests/  
      CreateProductRequest.cs  
      RegisterAnimalRequest.cs  
  
    Responses/  
      ProductResponse.cs  
      AnimalResponse.cs  
  
  Mappers/  
    ProductMapper.cs  
    AnimalMapper.cs  
  
  Presenters/  
    ProductPresenter.cs  
    ReportPresenter.cs  
  
  Middleware/  
    TenantResolutionMiddleware.cs  
    ExceptionHandlingMiddleware.cs  
    LoggingMiddleware.cs  
  
  Filters/  
    ValidationFilter.cs
```

Diagrama C4

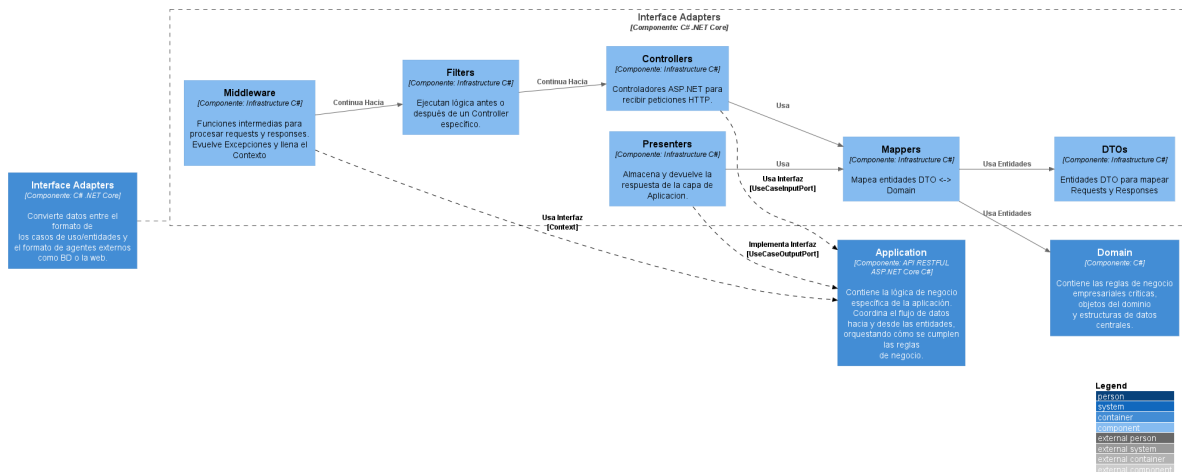


Figure 6: Diagrama C4 - Interface Adapters

Análisis de Cada Componente

Para el análisis se tendrá en cuenta este escenario:

POST /api/animals: Crear una vaca (Animal)

DTOs/

Requests/

CreateAnimalRequest.cs

```
namespace InterfaceAdapters.DTOs.Requests;

// Lo que recibe el API (HTTP)
public class CreateAnimalRequest
{
    public string Name { get; set; } = default!;
    public DateTime BirthDate { get; set; }
}
```

Responses/

AnimalResponse.cs

```
namespace InterfaceAdapters.DTOs.Responses;

// Lo que devuelve el API
public class AnimalResponse
{
    public Guid Id { get; set; }
    public string Name { get; set; } = default!;
}
```

Mappers/

AnimalMapper.cs

```
using Application.Modules.Animals.UseCases.CreateAnimal;
using InterfaceAdapters.DTOs.Requests;

namespace InterfaceAdapters.Mappers;

public static class AnimalMapper
{
    // HTTP → Application
    public static CreateAnimalCommand ToCommand(CreateAnimalRequest request)
    {
        return new CreateAnimalCommand
        {
            Name = request.Name,
            BirthDate = request.BirthDate
        };
    }
}
```

Presenters/

CreateAnimalPresenter.cs

```
using Application.Modules.Animals.UseCases.CreateAnimal;
using InterfaceAdapters.DTOs.Responses;

namespace InterfaceAdapters.Presenters;
```

```
// Implementa OutputPort
public class CreateAnimalPresenter : ICreateAnimalOutputPort
{
    public AnimalResponse? Response { get; private set; }

    public Task Ok(CreateAnimalOutput output)
    {
        Response = new AnimalResponse
        {
            Id = output.Id,
            Name = output.Name
        };

        return Task.CompletedTask;
    }
}
```

Controllers/

AnimalsController.cs

```
using Microsoft.AspNetCore.Mvc;
using InterfaceAdapters.DTOs.Requests;
using InterfaceAdapters.Mappers;
using InterfaceAdapters.Presenters;
using Application.Modules.Animals.UseCases.CreateAnimal;

namespace InterfaceAdapters.Controllers;

[ApiController]
[Route("api/[controller]")]
public class AnimalsController : ControllerBase
{
    private readonly ICreateAnimalInputPort _inputPort;
    private readonly CreateAnimalPresenter _presenter;

    public AnimalsController(
        ICreateAnimalInputPort inputPort,
        CreateAnimalPresenter presenter)
    {
        _inputPort = inputPort;
    }
}
```

```

        _presenter = presenter;
    }

    [HttpPost]
    public async Task<IActionResult> Create(CreateAnimalRequest request)
    {
        // 1. Mapear request → command
        var command = AnimalMapper.ToCommand(request);

        // 2. Ejecutar caso de uso
        await _inputPort.Handle(command);

        // 3. Devolver respuesta desde presenter
        return Ok(_presenter.Response);
    }
}

```

Middleware/

ExceptionHandlerMiddleware.cs

```

namespace InterfaceAdapters.Middleware;

public class ExceptionHandlingMiddleware
{
    private readonly RequestDelegate _next;

    public ExceptionHandlingMiddleware(RequestDelegate next)
    {
        _next = next;
    }

    public async Task InvokeAsync(HttpContext context)
    {
        try
        {
            await _next(context);
        }
        catch (Exception ex)
        {
            context.Response.StatusCode = 500;
        }
    }
}

```

```

        await context.Response.WriteAsJsonAsync(new
        {
            error = ex.Message
        });
    }
}
}

```

TenantResolutionMiddleware.cs

Se encarga de resolver el Tenant en cada request HTTP, usando el Context proporcionado por Infrastructure.

```

using Application.Common.Gateway.Context;

namespace InterfaceAdapters.Middleware;

public class TenantResolutionMiddleware
{
    private readonly RequestDelegate _next;
    private readonly string _baseDomain;

    public TenantResolutionMiddleware(
        RequestDelegate next,
        IConfiguration config)
    {
        _next = next;
        _baseDomain = config["MultiTenant:BaseDomain"]!;
    }

    public async Task InvokeAsync(
        HttpContext context,
        ITenantStore tenantStore,
        ITenantContext tenantContext)
    {
        // 1. Obtener host (subdominio)
        var host = context.Request.Host.Host;

        var slug = ExtractSlug(host);

        if (slug == null)
    }
}

```

```

    {
        context.Response.StatusCode = 404;
        await context.Response.WriteAsync("Tenant no encontrado");
        return;
    }

    // 2. Buscar tenant (Infrastructure)
    var tenant = await tenantStore.GetBySlugAsync(slug);

    if (tenant == null)
    {
        context.Response.StatusCode = 404;
        await context.Response.WriteAsync("Tenant inválido");
        return;
    }

    // 3. Guardar en contexto (clave )
    ((TenantContext)tenantContext)
        .SetTenant(tenant.Id, tenant.DatabaseName);

    // 4. Continuar pipeline
    await _next(context);
}

private string? ExtractSlug(string host)
{
    if (!host.EndsWith(".") + _baseDomain))
        return null;

    return host[..^(_baseDomain.Length + 1)];
}
}

```

Filters/

ValidationFilter.cs

Esta clase válida automáticamente modelos HTTP o DTO's antes de llegar al Controller.

```

using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.Filters;

```

```

namespace InterfaceAdapters.Filters;

public class ValidationFilter : IActionFilter
{
    public void OnActionExecuting(ActionExecutingContext context)
    {
        // Aqui valida que se cumplan las anotaciones en el DTO
        // como [Required], [Length], etc.
        // antes de llegar al Controller
        if (!context.ModelState.IsValid)
        {
            context.Result = new BadRequestObjectResult(context.ModelState);
        }
    }

    public void OnActionExecuted(ActionExecutedContext context) { }
}

```

VII. Infrastructure Layer

```

Infrastructure/
  Persistence/
    TenantCatalogDb.cs
    GanaderiaDbContext.cs

    Entities/
      ProductEntity.cs
      AnimalEntity.cs
      AppSettingEntity.cs
      FormFieldEntity.cs

    Mappers/
      ProductMapper.cs
      AnimalMapper.cs

    Configurations/
      (Fluent API EF Core)

```

```
Repositories/
    ProductRepository.cs
    AnimalRepository.cs

Tenant/
    ITenantDbContextFactory.cs
    TenantDbContextFactory.cs
    TenantContext.cs
    ITenantStore.cs
    TenantInfo.cs
    TenantStore.cs
    TenantProvisioningService.cs

Configuration/
    ConfigurationService.cs

Features/
    FeatureService.cs

Rules/
    RuleEngineService.cs

Caching/
    AppCacheService.cs

Context/
    RequestContext.cs

Identity/
    CurrentUserService.cs

Auditing/
    AuditSaveChangesInterceptor.cs

Time/
    DateTimeProvider.cs

External/
    SenasaClient.cs
    WhatsAppService.cs
    PowerBIExporter.cs
```


Diagrama C4

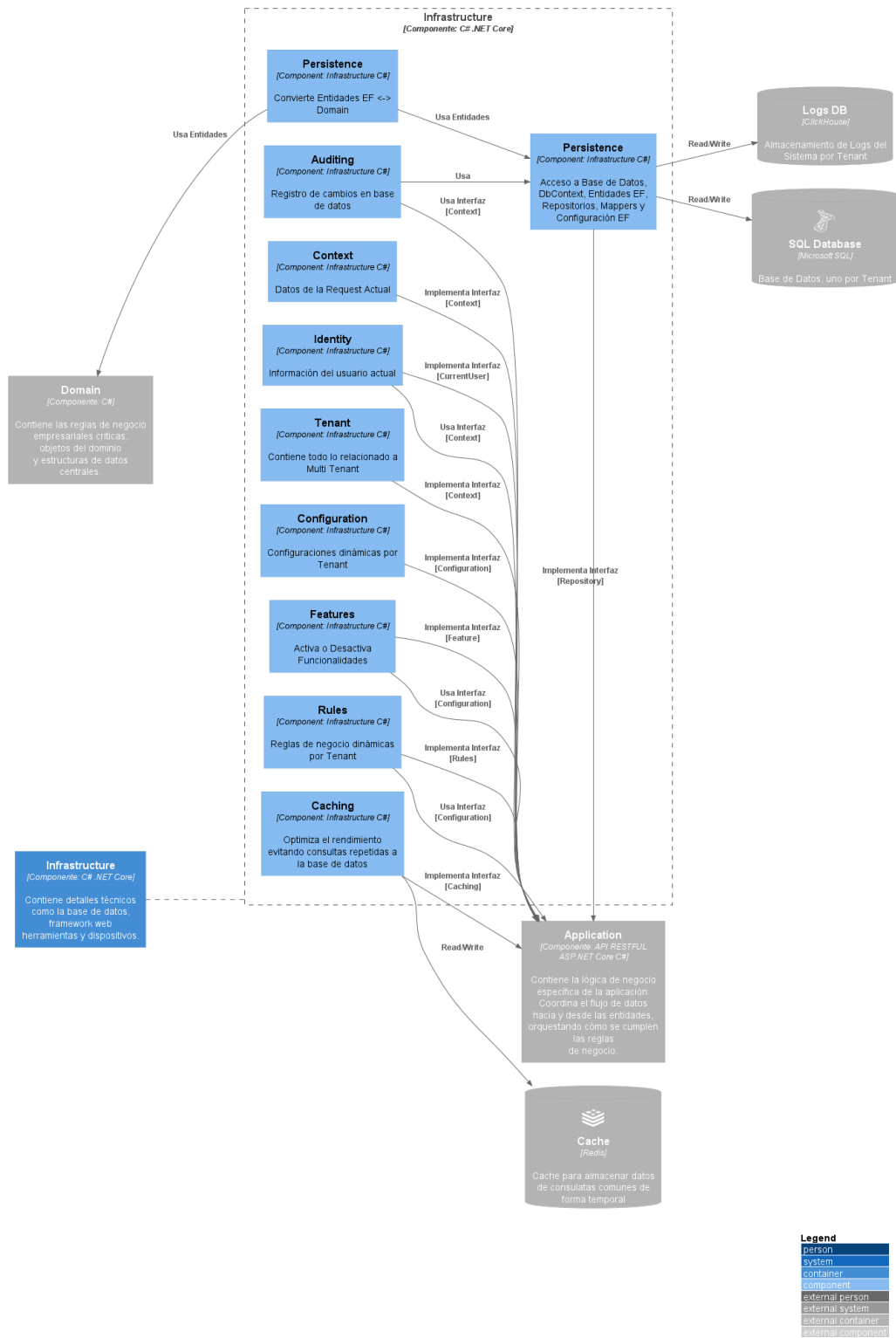


Figure 7: Diagrama C4 - Domain

Análisis de Cada Componente

Persistence/

GanaderiaDbContext.cs

Se encarga de definir la conexión con la base de datos.

```
// Infrastructure/
// Persistence/GanaderiaDbContext.cs
using Microsoft.EntityFrameworkCore;
using Infrastructure.Persistence.Entities;

namespace Infrastructure.Persistence;

public class GanaderiaDbContext : DbContext
{
    // Registra la Clase CowEntity como Entity
    public DbSet<CowEntity> Cows => Set<CowEntity>();

    public GanaderiaDbContext(DbContextOptions<GanaderiaDbContext> options,
        AuditSaveChangesInterceptor audit)
        : base(options)
    {
        _audit = audit;
    }

    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
    {
        // Aquí se registra el interceptor para la auditoria en BD
        optionsBuilder.AddInterceptors(_audit);
    }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        // Aplica configuración Fluent API
        modelBuilder.ApplyConfigurationsFromAssembly(typeof(GanaderiaDbContext).Assembly);
    }
}
```

TenantCatalogDb.cs

Este se diferencia del Context anterior ya que este Context sirve para guardar la informacion de un tenant.

```
// Infrastructure/  
// Persistence/TenantCatalogDb.cs  
using Microsoft.EntityFrameworkCore;  
  
namespace Infrastructure.Persistence;  
  
public class TenantCatalogDb : DbContext  
{  
    public DbSet<TenantEntity> Tenants => Set<TenantEntity>();  
  
    public TenantCatalogDb(DbContextOptions<TenantCatalogDb> options)  
        : base(options) { }  
}  
  
// Entidad ORM  
public class TenantEntity  
{  
    public Guid Id { get; set; }  
    public string Slug { get; set; } = default!;  
    public string DatabaseName { get; set; } = default!;  
    public bool IsActive { get; set; }  
}
```

Entities/

CowEntity.cs

```
namespace Infrastructure.Persistence.Entities;  
  
// Entidad de base de datos (NO es la del dominio)  
public class CowEntity  
{  
    public Guid Id { get; set; }  
    public string Name { get; set; } = default!;  
    public DateTime BirthDate { get; set; }  
}
```

Mappers/

CowMapper.cs

Esta clase se encargará de mapear las entidades de dominio a las entidades ORM y viceversa.

```
using Domain.Entities;
using Infrastructure.Persistence.Entities;

namespace Infrastructure.Persistence.Mappers;

public static class CowMapper
{
    // Domain → Entity
    public static CowEntity ToEntity(Animal animal)
    {
        return new CowEntity
        {
            Id = animal.Id,
            Name = animal.Name,
            BirthDate = animal.BirthDate
        };
    }

    // Entity → Domain
    public static Animal ToDomain(CowEntity entity)
    {
        return new Animal(entity.Id, entity.Name, entity.BirthDate);
    }
}
```

Configurations/

Aquí se encuentran las clases que se encargan de configurar las Entidades ORM, permitiendo añadir restricciones (nullable, unique, max length, etc.), relaciones (1:1 , 1:n , n:m), índices, etc.

CowConfiguration.cs

```

using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;
using Infrastructure.Persistence.Entities;

namespace Infrastructure.Persistence.Configurations;

public class CowConfiguration : IEntityTypeConfiguration<CowEntity>
{
    public void Configure(EntityTypeBuilder<CowEntity> builder)
    {
        // Nombre de tabla en BD
        builder.ToTable("Cows");

        // Definir llave primaria
        builder.HasKey(x => x.Id);

        // Añadir restricciones a campo Name
        builder.Property(x => x.Name)
            .IsRequired()
            .HasMaxLength(100);

        // Añadir restricciones a campo BirthDate
        builder.Property(x => x.BirthDate)
            .IsRequired();
    }
}

```

Repositories/

Los repositorios nos permitirán interactuar con nuestros datos almacenados en la base de datos por medio de código o consultas Raw SQL, Recuerda que este repositorio implementa la Interfaz definida en Application:

AnimalRepository.cs

```

using Domain.Entities;
using Application.Modules.Animals.Interfaces;
using Infrastructure.Persistence.Mappers;

namespace Infrastructure.Persistence.Repositories;

```

```
// Implementa interfaz de Application
public class AnimalRepository : IAnimalRepository
{
    // Se obtiene DbContext del Tenant actual
    private readonly ITenantDbContextFactory _factory;

    public AnimalRepository(ITenantDbContextFactory factory)
    {
        _factory = factory;
    }

    public async Task AddAsync(Animal animal)
    {
        using var db = _factory.CreateDbContext();

        var entity = CowMapper.ToEntity(animal);

        db.Cows.Add(entity);

        await db.SaveChangesAsync();
    }

    public async Task<List<Animal>> GetAllAsync()
    {
        using var db = _factory.CreateDbContext();

        var entities = db.Cows.ToList();

        return entities.Select(CowMapper.ToDomain).ToList();
    }
}
```

Tenant/

ITenantDbContextFactory.cs

Interfaz para implementar patron Factory, se usará para llamar al DbContext correcto sin tener que cambiar la dependencia cada vez.

```
using Infrastructure.Persistence;

public interface ITenantDbContextFactory
```

```
{
    GanaderiaDbContext CreateDbContext();
}
```

TenantDbContextFactory.cs

Implementa la interfaz `ITenantDbContextFactory`, y será usado para conectarse a la base de datos correcta cuando un usuario ingrese.

```
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Infrastructure.Persistence;
using Application.Common.Gateway.Context;

namespace Infrastructure.Tenant;

public class TenantDbContextFactory : ITenantDbContextFactory
{
    // Necesitamos la info del Tenant para conectarnos a la BD
    // correcta
    private readonly ITenantContext _tenant;
    private readonly IConfiguration _config;

    public TenantDbContextFactory(ITenantContext tenant, IConfiguration config)
    {
        _tenant = tenant;
        _config = config;
    }

    public GanaderiaDbContext CreateDbContext()
    {
        var template = _config.GetConnectionString("TenantTemplate")!;
        var conn = template.Replace("{DATABASE}", _tenant.DatabaseName);

        var options = new DbContextOptionsBuilder<GanaderiaDbContext>()
            .UseSqlServer(conn)
            .Options;

        return new GanaderiaDbContext(options);
    }
}
```

TenantContext.cs

Esta clase se usará como almacén, para guardar los datos del tenant actual y poder acceder después a ellos cuando se requiera, por ejemplo, cuando se necesite conectar a una determinada base de datos.

```
using Application.Common.Gateway.Context;

namespace Infrastructure.Tenant;

public class TenantContext : ITenantContext
{
    public Guid TenantId { get; private set; }
    public string DatabaseName { get; private set; } = "";

    public void SetTenant(Guid id, string dbName)
    {
        TenantId = id;
        DatabaseName = dbName;
    }
}
```

ITenantStore.cs

```
namespace Infrastructure.Tenant;

public interface ITenantStore
{
    Task<TenantInfo?> GetBySlugAsync(string slug);
}
```

TenantInfo.cs

Este DTO servirá para mapear la información del Tenant actual obtenido por medio de la clase TenantStore.cs.

```
namespace Infrastructure.Tenant;

// DTO interno de infraestructura
public class TenantInfo
{
    public Guid Id { get; set; }
```

```

    public string Slug { get; set; } = default!;
    public string DatabaseName { get; set; } = default!;
    public bool IsActive { get; set; }
}

```

TenantStore.cs

Sirve para obtener la información del **Tenant**, ya sea por base de datos o cache.

```

using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Caching.Memory;
using Infrastructure.Persistence;

namespace Infrastructure.Tenant;

public class TenantStore : ITenantStore
{
    private readonly TenantCatalogDb _db;
    private readonly IMemoryCache _cache;

    public TenantStore(TenantCatalogDb db, IMemoryCache cache)
    {
        _db = db;
        _cache = cache;
    }

    public async Task<TenantInfo?> GetBySlugAsync(string slug)
    {
        var cacheKey = $"tenant:{slug}";

        // Intenta cache
        if (_cache.TryGetValue(cacheKey, out TenantInfo cached))
            return cached;

        // Busca en DB
        var tenant = await _db.Tenants
            .AsNoTracking()
            .FirstOrDefaultAsync(t => t.Slug == slug);

        if (tenant == null || !tenant.IsActive)
            return null;
    }
}

```

```

        // Almacenar info del Tenant
        // en DTO
        var result = new TenantInfo
        {
            Id = tenant.Id,
            Slug = tenant.Slug,
            DatabaseName = tenant.DatabaseName,
            IsActive = tenant.IsActive
        };

        // Guarda en cache
        _cache.Set(cacheKey, result, TimeSpan.FromMinutes(5));

        return result;
    }
}

```

TenantProvisioningService.cs

Esta clase servirá para crear un nuevo Tenant.

```

using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Infrastructure.Persistence;

namespace Infrastructure.Tenant;

public class TenantProvisioningService
{
    private readonly TenantCatalogDb _catalogDb;
    private readonly IConfiguration _config;

    public TenantProvisioningService(
        TenantCatalogDb catalogDb,
        IConfiguration config)
    {
        _catalogDb = catalogDb;
        _config = config;
    }

    public async Task ProvisionAsync(CreateTenantDto dto)
    {

```

```

// Generar nombre de base de datos
var dbName = $"Ganaderia_{dto.Slug.Replace("-", "_")}_Db";

// Registrar Tenant en catálogo
var tenant = new TenantEntity
{
    Id = Guid.NewGuid(),
    Slug = dto.Slug,
    DatabaseName = dbName,
    IsActive = true
};

_catalogDb.Tenants.Add(tenant);
await _catalogDb.SaveChangesAsync();

// Crear DB dinámica
var template = _config.GetConnectionString("TenantTemplate");
var conn = template.Replace("{DATABASE}", dbName);

var options = new DbContextOptionsBuilder<GanaderiaDbContext>()
    .UseSqlServer(conn)
    .Options;

await using var db = new GanaderiaDbContext(options);

// Aplicar migraciones automáticamente
await db.Database.MigrateAsync();

// Seed inicial (opcional)
await SeedAsync(db);
}

private async Task SeedAsync(GanaderiaDbContext db)
{
    if (!db.Cows.Any())
    {
        db.Cows.Add(new Persistence.Entities.CowEntity
        {
            Id = Guid.NewGuid(),
            Name = "Vaca Inicial",
            BirthDate = DateTime.UtcNow
        });
    }
}

```

```

        await db.SaveChangesAsync();
    }
}

// Este DTO servira para pasar los datos necesarios
// para la creacion del Tenant
// puede ir en InterfaceAdapters y luego mapearse hacia un DTO
// en Application donde habra un UseCase para crear el tenant
// Pero se debe crear un Gateway para poder interactuar con la clase
// sin romper la arquitectura
// por simplicidad se puede mantener en /Tenant/CreateTenantDto.cs
public class CreateTenantDto
{
    public string Slug { get; set; } = default!;
    public string DisplayName { get; set; } = default!;
    public string Email { get; set; } = default!;
}

```

Configuration/

ConfigurationService.cs

Sirve para obtener las configuraciones actuales que cada Tenant pueda tener (esto se guarda en BD), cosas como Días Estándar de Celo, Campos Obligatorios, Idioma, Unidades (litros o kg), Litros de Leche mínima, etc. Los resultados pueden ser usados por la capa de Application.

```

using Application.Common.Gateway.Configuration;

namespace Infrastructure.Configuration;

public class ConfigurationService : IConfigurationService
{
    private readonly ITenantDbContextFactory _factory;

    public ConfigurationService(ITenantDbContextFactory factory)
    {
        _factory = factory;
    }

    public async Task<T?> GetValueAsync<T>(string key)

```

```

{
    using var db = _factory.CreateDbContext();

    var setting = db.Set<AppSettingEntity>()
        .AsNoTracking()
        .FirstOrDefault(x => x.Key == key);

    if (setting == null)
        return default;

    return (T)Convert.ChangeType(setting.Value, typeof(T));
}
}

```

Features/

FeatureService.cs

Esto controla features activas o inactivas por tenant, por ejemplo, un Cliente A puede tener un modulo activado y otro cliente B no. Devuelve un booleano y es usada por la capa de Application.

```

using Application.Common.Gateway.Features;
using Microsoft.EntityFrameworkCore;

namespace Infrastructure.Features;

public class FeatureService : IFeatureService
{
    private readonly ITenantDbContextFactory _factory;

    public FeatureService(ITenantDbContextFactory factory)
    {
        _factory = factory;
    }

    public async Task<bool> IsEnabledAsync(string feature)
    {
        using var db = _factory.CreateDbContext();

        var result = await db.Set<FeatureEntity>()
            .AsNoTracking()

```

```

        .FirstOrDefaultAsync(f => f.Name == feature);

        return result?.IsEnabled ?? false;
    }
}

// Debe tener su Entity ORM en EF Core
public class FeatureEntity
{
    public Guid Id { get; set; }
    public string Name { get; set; } = default!;
    public bool IsEnabled { get; set; }
}

```

Rules/

RuleEngineService.cs

Esto se diferencia de **Configuration**, ya que aquí se ejecutan reglas de negocio dinámicas, las reglas o lógica pueden guardarse en base de datos o escribirse como código. Se puede implementar usando librerías como **NRules**.

Se pueden usar **Configuration** y **Rules** en conjunto, una para obtener la configuración establecida en el Tenant y otra para evaluar si los datos que se ingresan son correctos. Por ejemplo, si tenemos una configuración **milk.min.liters** que nos diga que cantidad de leche se considera baja y una regla **milk.low.production** que se encargue de evaluar si el input del usuario es realmente bajo o no según lo que dice el valor de la configuración.

```

using Application.Common.Gateway.Rules;

namespace Infrastructure.Rules;

public class RuleEngineService : IRuleEngineService
{
    public Task<bool> EvaluateAsync(string rule, object input)
    {
        // Por ejemplo aqui se evalua si la cantidad
        // de leche ingresada es bajo o no
        // devuelve true o false
        if (rule == "milk.low.production")
        {
            dynamic data = input;

```

```

        return Task.FromResult(data.Liters < 10);
    }

    return Task.FromResult(true);
}
}

```

Caching/

AppCacheService.cs

```

using Microsoft.Extensions.Caching.Memory;
using Application.Common.Gateway.Caching;

namespace Infrastructure.Caching;

public class AppCacheService : IAppCacheService
{
    private readonly IMemoryCache _cache;

    public AppCacheService(IMemoryCache cache)
    {
        _cache = cache;
    }

    public async Task<T> GetOrCreateAsync<T>(
        string key,
        Func<Task<T>> factory,
        TimeSpan expiration)
    {
        if (_cache.TryGetValue(key, out T value))
            return value;

        value = await factory();

        _cache.Set(key, value, expiration);

        return value;
    }
}

```

Context/

RequestContext.cs

```
using Application.Common.Gateway.Context;
using Microsoft.AspNetCore.Http;
using System.Security.Claims;

namespace Infrastructure.Context;

public class RequestContext : IRequestContext
{
    private readonly IHttpContextAccessor _http;

    public RequestContext(IHttpContextAccessor http)
    {
        _http = http;
    }

    public string? UserId =>
        _http.HttpContext?.User?.FindFirst("sub")?.Value;

    public string? UserEmail =>
        _http.HttpContext?.User?.FindFirst(ClaimTypes.Email)?.Value;

    public string? CorrelationId =>
        _http.HttpContext?.TraceIdentifier;

    public string? Path =>
        _http.HttpContext?.Request?.Path.Value;
}
```

Identity/

CurrentUserService.cs

```
using Application.Common.Gateway.Identity;
using Application.Common.Gateway.Context;

namespace Infrastructure.Identity;

public class CurrentUserService : ICurrentUserService
```

```

{
    private readonly IRequestContext _ctx;

    public CurrentUserService(IRequestContext ctx)
    {
        _ctx = ctx;
    }

    public string? UserId => _ctx.UserId;
}

```

Auditing/

AuditSaveChangesInterceptor.cs

```

using Microsoft.EntityFrameworkCore.Diagnostics;
using Microsoft.EntityFrameworkCore;
using Application.Common.Gateway.Context;

namespace Infrastructure.Auditing;

public class AuditSaveChangesInterceptor : SaveChangesInterceptor
{
    private readonly ITenantContext _tenant;
    private readonly IRequestContext _request;

    public AuditSaveChangesInterceptor(
        ITenantContext tenant,
        IRequestContext request)
    {
        _tenant = tenant;
        _request = request;
    }

    public override async ValueTask<InterceptionResult<int>> SavingChangesAsync(
        DbContextEventData eventData,
        InterceptionResult<int> result,
        CancellationToken cancellationToken = default)
    {
        var ctx = eventData.Context!;
    }
}

```

```

foreach (var entry in ctx.ChangeTracker.Entries())
{
    if (entry.State == EntityState.Added ||
        entry.State == EntityState.Modified ||
        entry.State == EntityState.Deleted)
    {
        // Evitar auditar la propia tabla de auditoria
        // evitando asi bucles
        if (entry.Entity is AuditEntry)
            continue;

        var audit = new AuditEntry
        {
            TenantId = _tenant.TenantId,
            UserId = _request.UserId,
            Entity = entry.Entity.GetType().Name,
            Operation = entry.State.ToString(),
            Timestamp = DateTime.UtcNow
        };

        context.Set<AuditEntry>().Add(audit);
    }
}

return await base.SavingChangesAsync(eventData, result, cancellationTokens);
}
}

```

Time/

DateTimeProvider.cs

```

namespace Infrastructure.Time;

public class DateTimeProvider : IDateTimeProvider
{
    public DateTime.UtcNow => DateTime.UtcNow;
}

```

External/

SenasaClient.cs

```
namespace Infrastructure.External;

public class SenasaClient
{
    private readonly HttpClient _http;

    public SenasaClient(HttpClient http)
    {
        _http = http;
    }

    public async Task SendAsync(object data)
    {
        await _http.PostAsJsonAsync("/senasa", data);
    }
}
```

FLUJO COMPLETO

```
HTTP Request
→ Middleware (Tenant)
→ Controller
→ UseCase (Application)
→ Domain
→ Repository (Infrastructure)
→ DB
```

CONCLUSIÓN

Arquitectura desacoplada Multi-tenant Altamente configurable Escalable
